pfi

Python for Fixed Income

COURSE 10 · IMPLEMENTING RAG

BM25 and Hybrid Search

WEEK 2 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: BM25 and hybrid search

Three ways to rank one 10-K

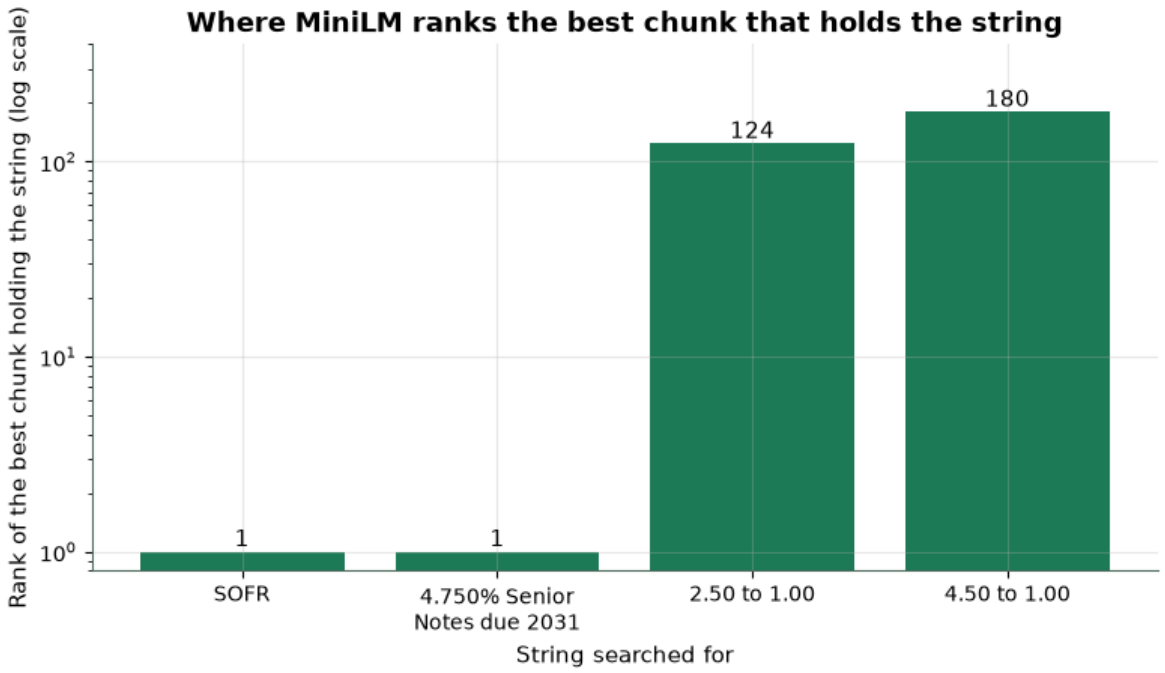
Chunks	1,158 at 512/64
MiniLM	ranks by meaning (day 4)
BM25	ranks by shared words
Hybrid	fuses the two lists
Model	none today; nothing is sent

Today

1. Keyword search is not dead
2. BM25 by hand
3. BM25 on the 10-K; the library; k1 and b
4. Counting hits; what each misses
5. Reciprocal rank fusion
6. Hybrid; recall against k; depth and constant

Count on development questions; name the rule; choose nothing yet.

Keyword search is not dead



Chunks holding the string

SOFR

17 chunks; best rank 1 of 1,158; 10 in the top 10

4.750% Senior Notes due 2031

14 chunks; best rank 1 of 1,158; 4 in the top 10

2.50 to 1.00

3 chunks; best rank 124 of 1,158; 0 in the top 10

4.50 to 1.00

3 chunks; best rank 180 of 1,158; 0 in the top 10

arc04's page 38 chunk with 4.50 to 1.00: MiniLM ranks it 54 of 1,158 for the question as worded.

How BM25 works

The problem	A search by meaning cannot insist on an exact string, and a desk question often hangs on one
The method	Tokens; a large weight for a token few chunks hold; a count that levels off; a discount for a long chunk; sum over the question's tokens
You control	k1 (how fast the count levels off) and b (how much a long chunk is discounted); the tokenizer sets what counts as a term
What comes out	One number per chunk, used only to order them; a score of 12 means nothing on its own

BM25 by hand: senior notes 2031

Three parts

```
idf = ln((N - n + 0.5) / (n + 0.5) + 1)

count = f (k1 + 1) / (f + k1 (1 - b + b dl / avgdl))

weight = idf x count; score = sum of weights

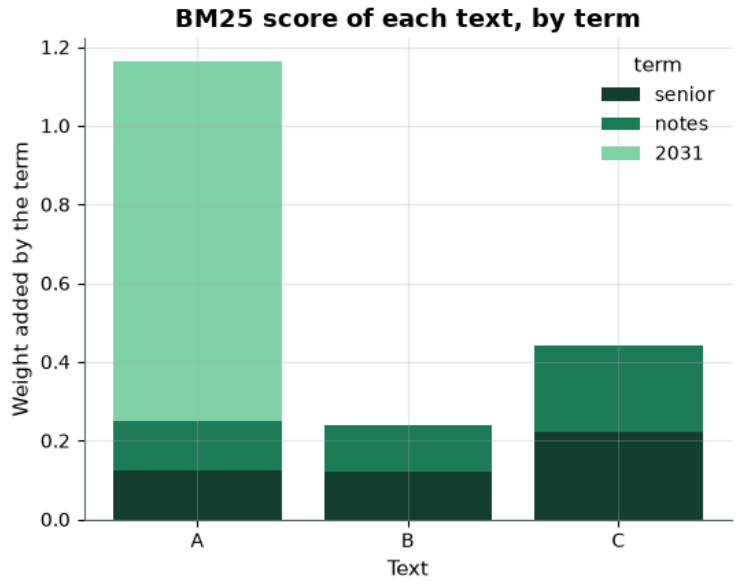
k1 = 1.5, b = 0.75; length in tokens
```

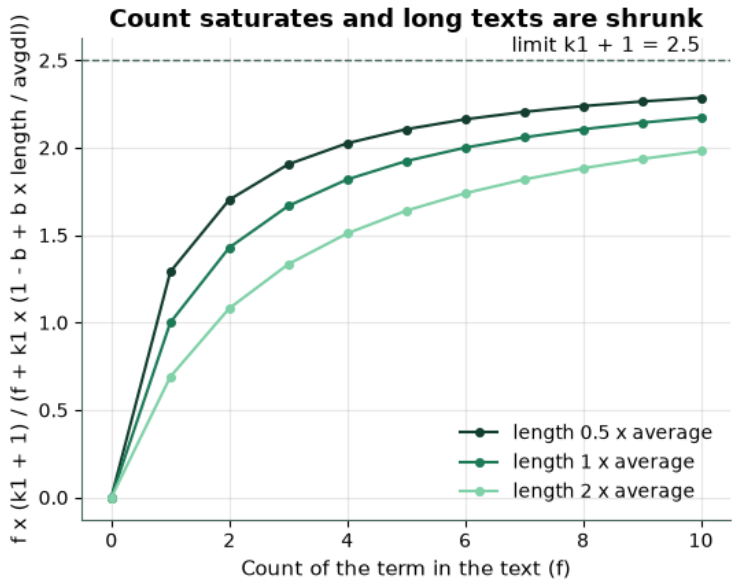
scores: A 1.1634, B 0.2393, C 0.4410

text	term	f	n	idf	dl	weight
A	senior	1	3	0.1335	12	0.1245
B	senior	1	3	0.1335	13	0.1196
C	senior	2	3	0.1335	6	0.2205
A	notes	1	3	0.1335	12	0.1245
B	notes	1	3	0.1335	13	0.1196
C	notes	2	3	0.1335	6	0.2205
A	2031	1	1	0.9808	12	0.9145
B	2031	0	1	0.9808	13	0.0000
C	2031	0	1	0.9808	6	0.0000

2031 is in one text, so it does the work; repeats and length are discounted.

Terms add up, counts level off





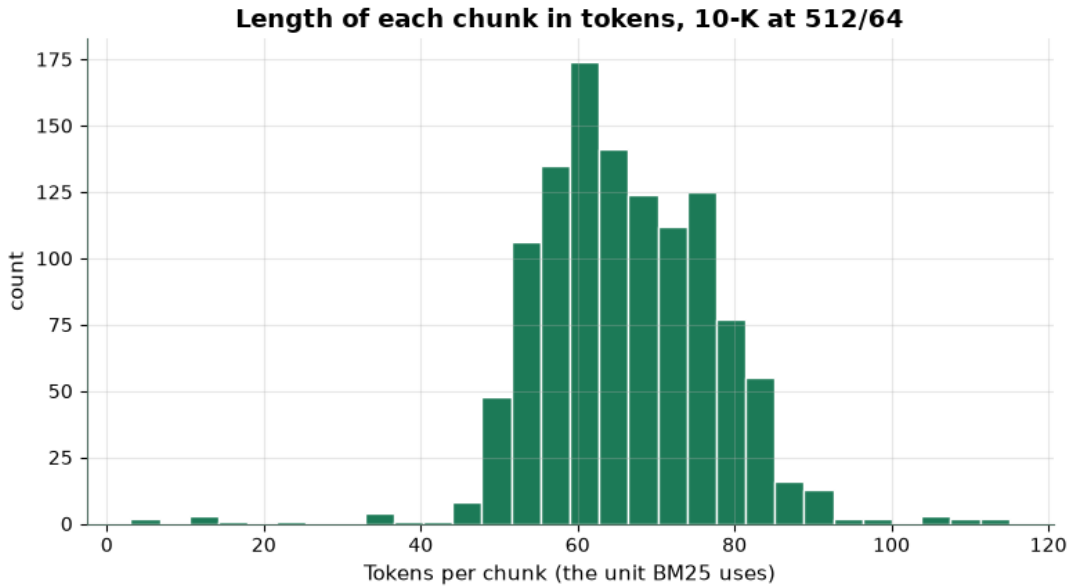
Where BM25 comes from

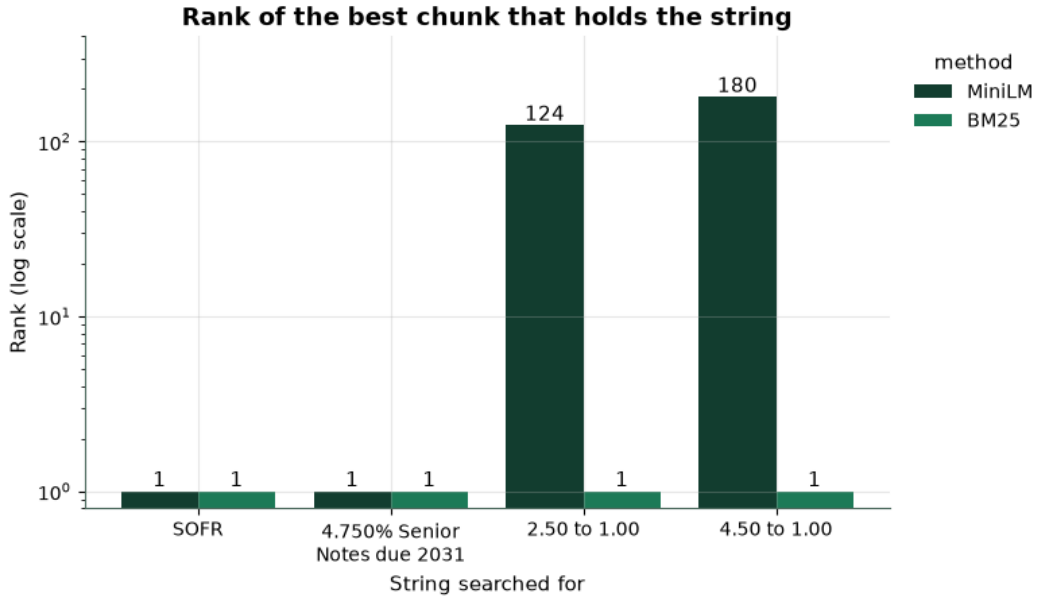
Robertson and Zaragoza, The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval 4(1-2), pp. 1-174, published 17 September 2009.

Crossref's record of DOI 10.1561/1500000019, read 13 October 2026. The paper itself was not read.

A term repeated ten times is not ten times a term seen once; a long text is shrunk.

BM25 on the 10-K





3-115

tokens per chunk

66.06

mean (avgdl)

426.8

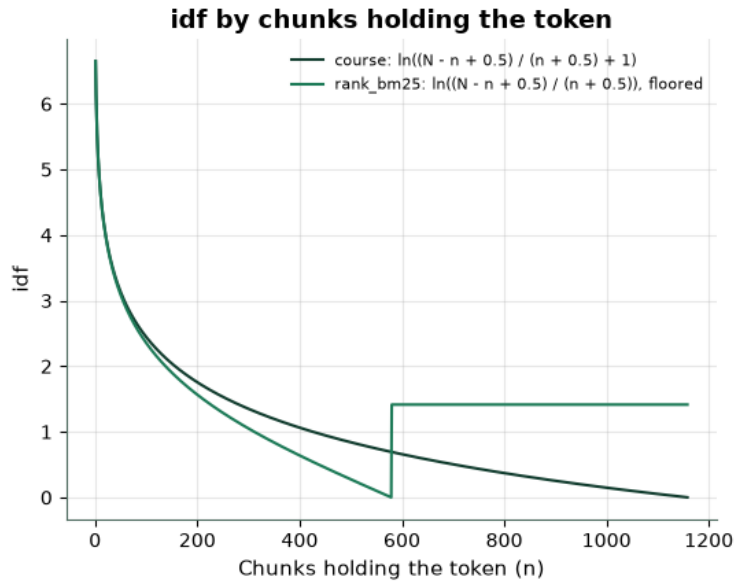
characters, mean

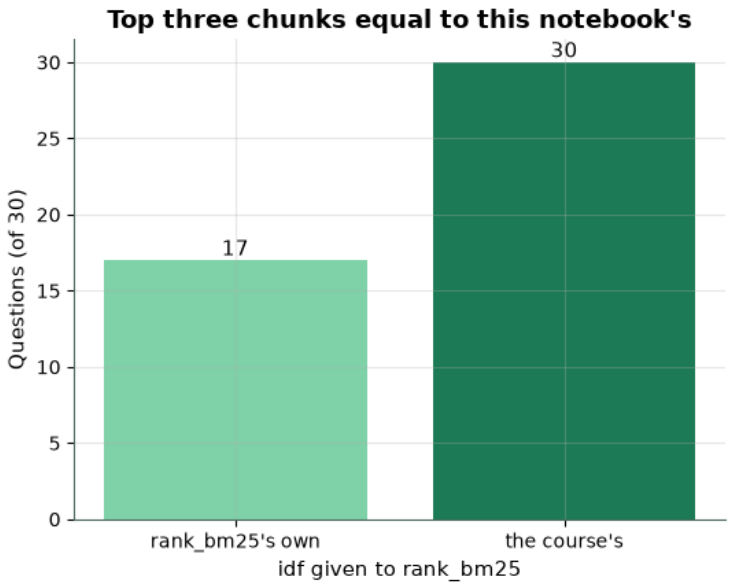
975

chunks scoring 0

Length in tokens throughout; BM25 ranks a chunk with each covenant level first, MiniLM does not.

How the rank_bm25 library differs





30 questions, top 3 chunks

own idf: 17 of 30 equal

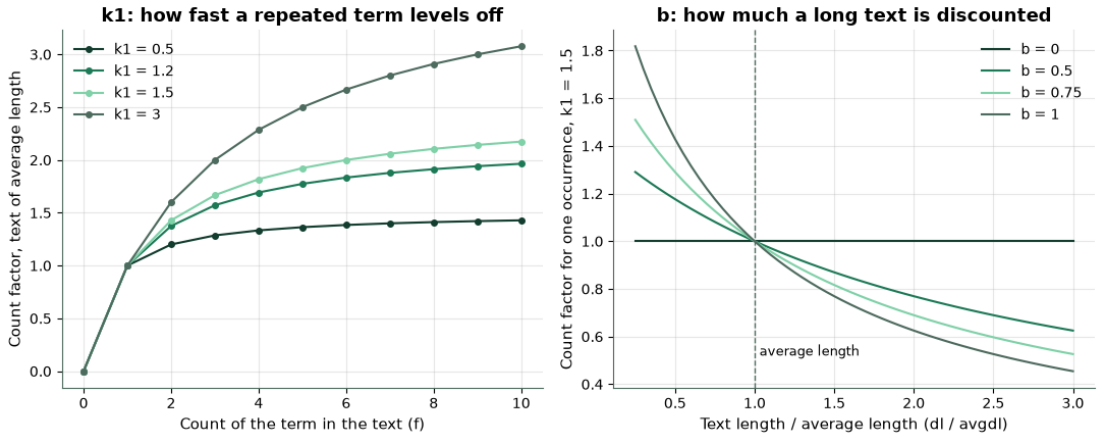
largest score gap 12.9

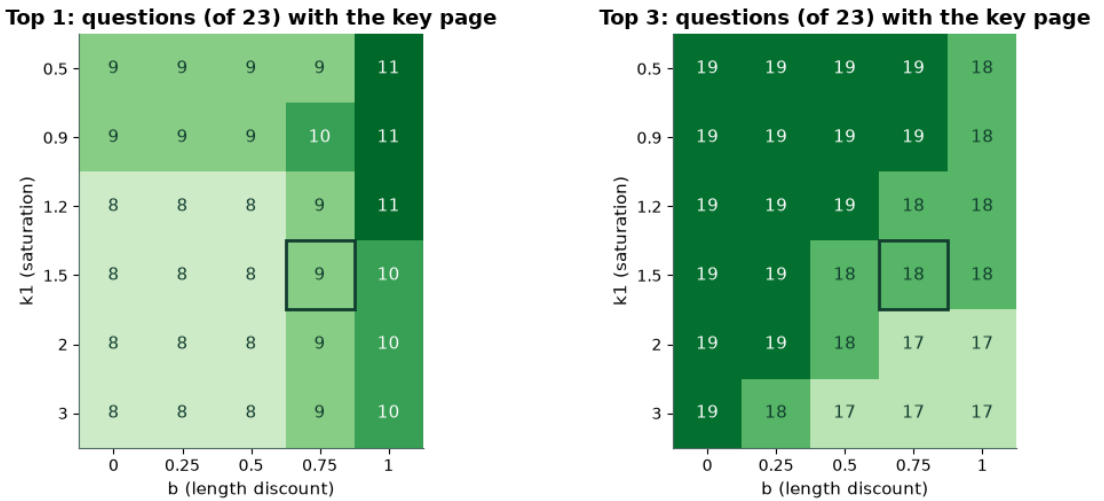
course idf put in: 30 of 30

scores equal within 1e-12

token	chunks	course idf	rank_bm25 idf					
the	1008	0.1391	1.4143		4.750	42	3.3058	3.2684
of	1023	0.1243	1.4143		sofr	17	4.1931	4.1779
senior	78	2.6922	2.6221					

k1 and b are settings: drawn, then swept



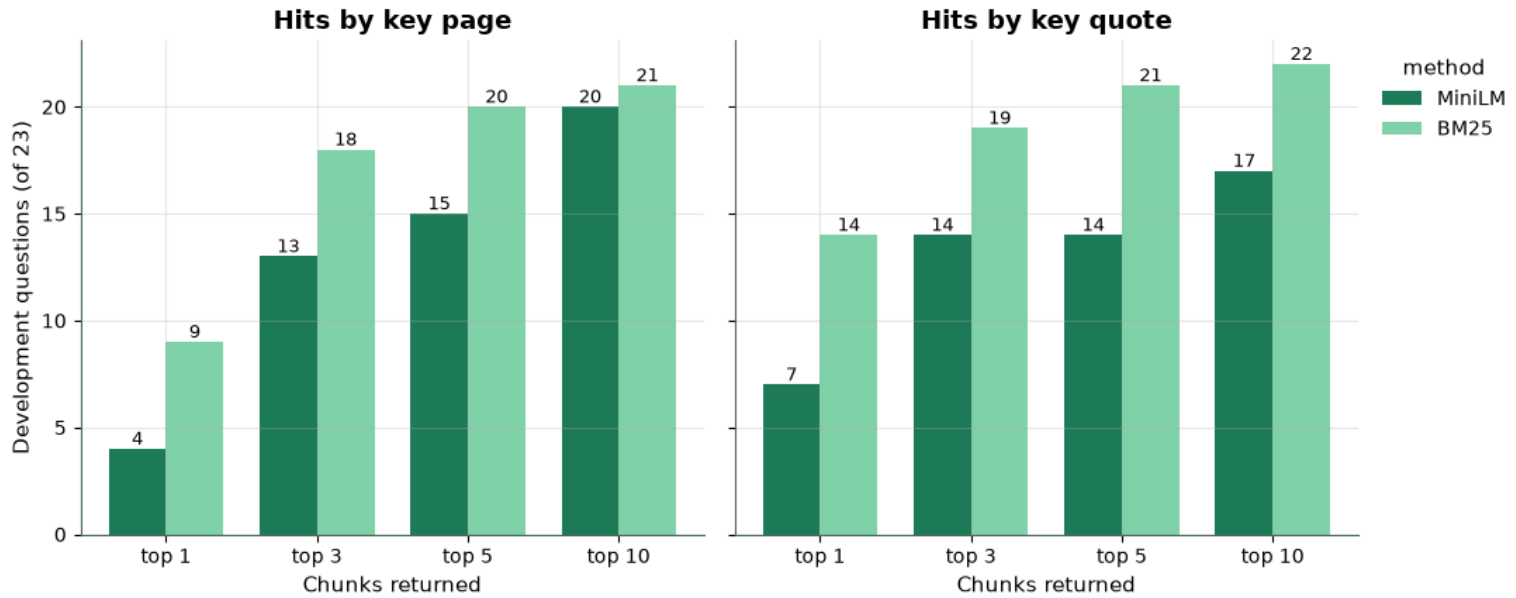


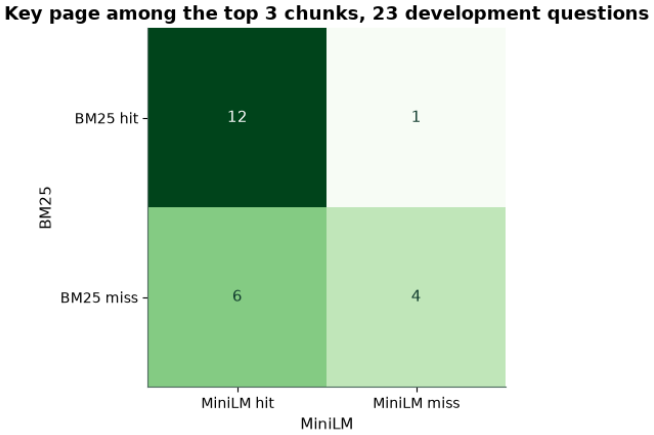
Top 3: 17 to 19 of 23 over 30 settings; 18 at the course's k1 = 1.5, b = 0.75 (ringed)

Top 1: 8 to 11; the most at b = 1, where the top 3 has the fewest

Read a grid for a plateau, not a peak; one or two questions on 23 is not a reason to move a setting.

MiniLM and BM25 on 23 development questions

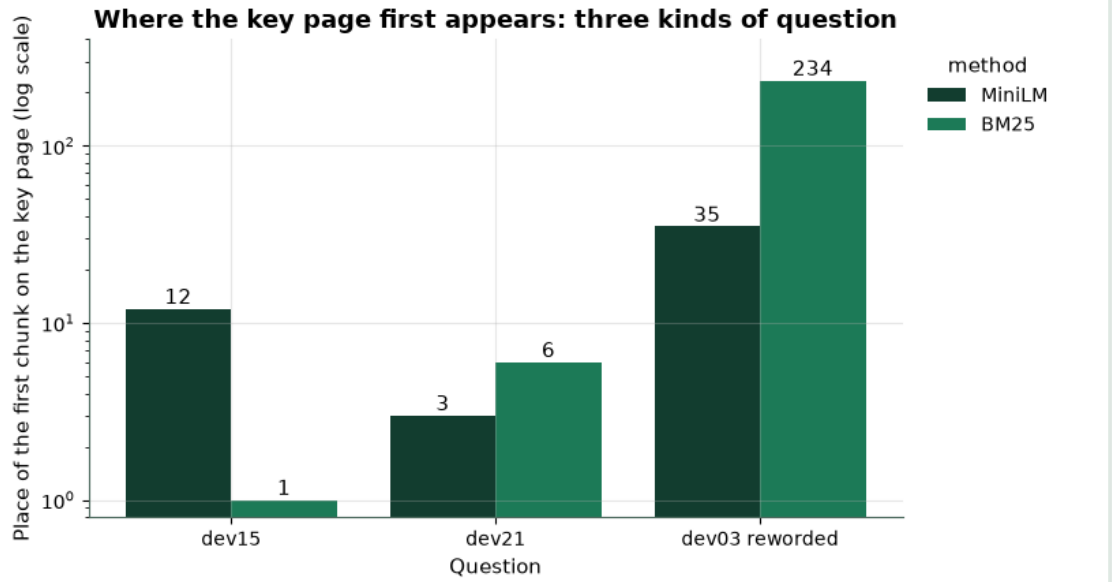


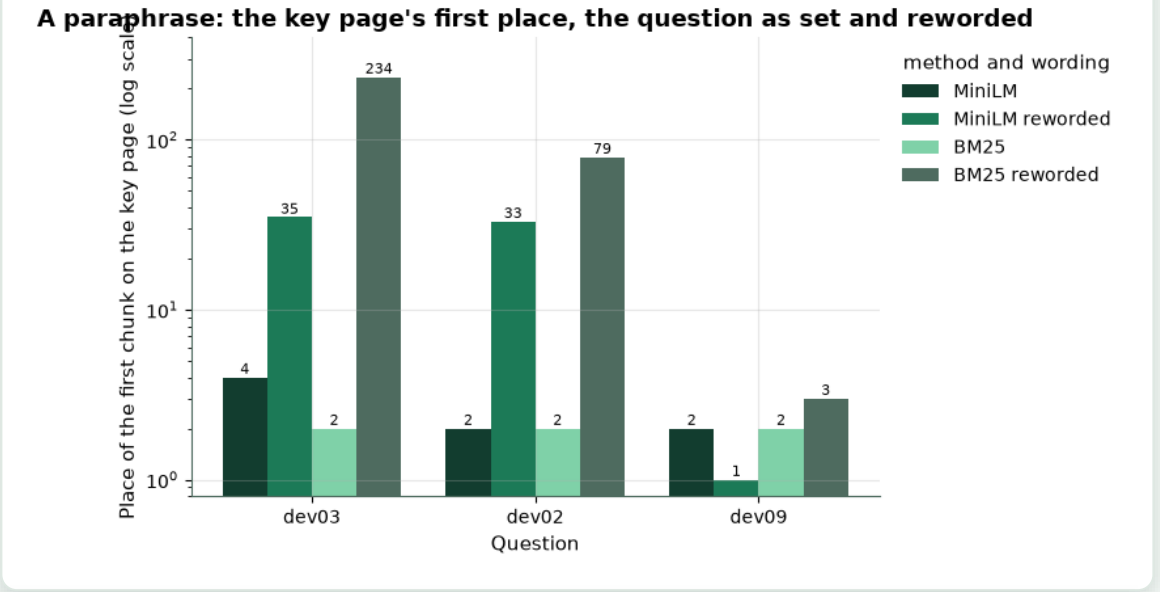


method	rule	top 1	top 3	top 5	top 10
MiniLM	key page	4	13	15	20
MiniLM	key quote	7	14	14	17
BM25	key page	9	18	20	21
BM25	key quote	14	19	21	22

Page rule and quote rule, named each time; a description of these questions, not a ranking of methods.

What each method misses





question	kind	MiniLM	BM25	token	chunks	idf
dev15	a figure beside a rare name	12	1	fanduel	55	3.04
dev21	a defined term	3	6	ebitdar	30	3.64
dev03 reworded	a paraphrase	35	234	revolver	0	7.75

A rare name is BM25's; a defined term is shared by 30 chunks; "revolver" is in 0 chunks, so a paraphrase hurts BM25 most.

How reciprocal rank fusion works

The problem	Two lists order the same chunks differently, and a BM25 score and a cosine cannot be added
The method	Forget the scores; each list gives a chunk 1 / (c + place), a missing chunk gets nothing, sum, and order by the sum
You control	The constant c (how much the top places are worth) and how many chunks each list contributes
What comes out	One fused list, best first; a chunk high in both lists beats a chunk first in only one

Reciprocal rank fusion

A place in a list is worth 1 / (c + place)

Sum over the lists a chunk is in

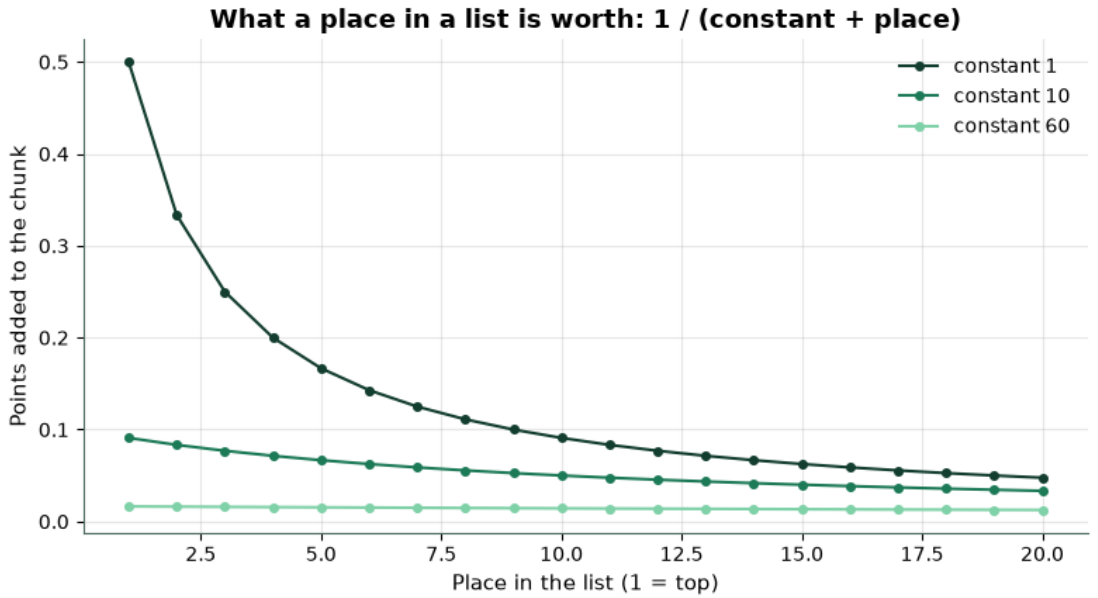
A chunk missing from a list adds nothing

c = 60; ties by chunk id

By hand: a b c and b c d

```
place 1: b  0.032522
place 2: c  0.032002
place 3: a  0.016393
place 4: d  0.015873
```

dev21: 34 chunks in the two top-20 lists; first on the key's page: fused 1, MiniLM 3, BM25 6.



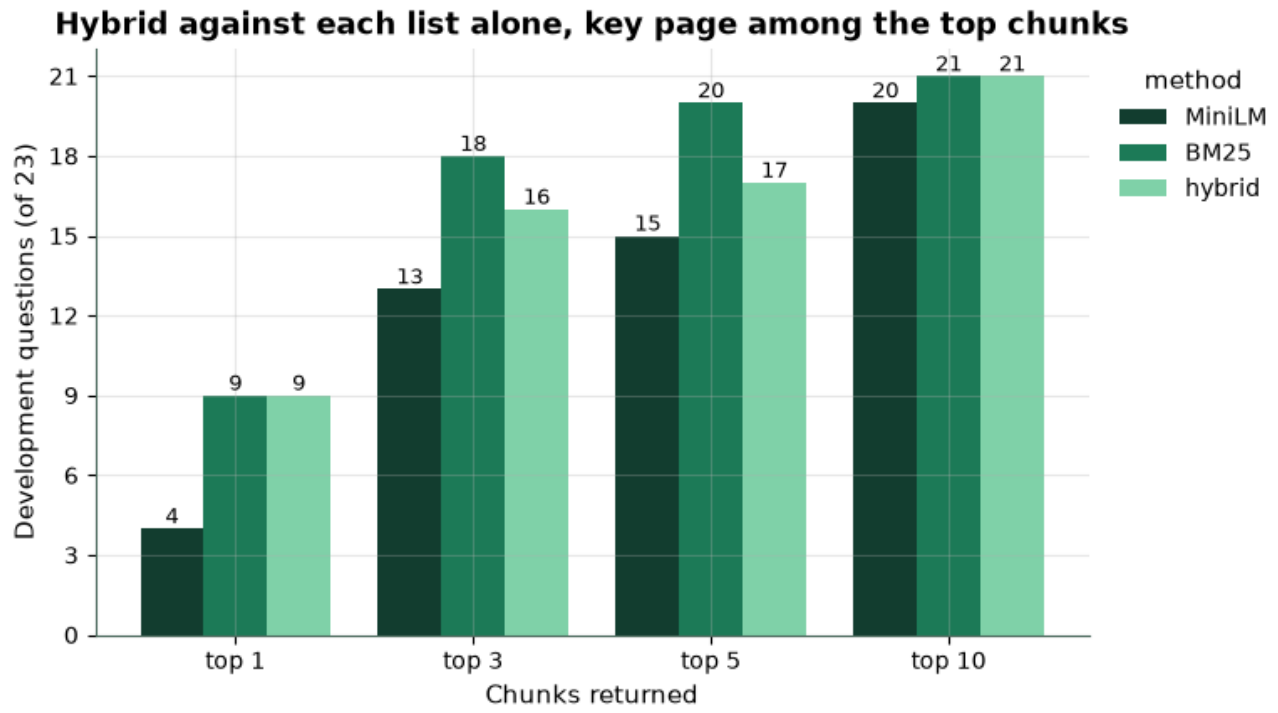
Cormack, Clarke, Buettcher (SIGIR 2009)

Reciprocal rank fusion outperforms condorcet and individual rank learning methods, pp. 758-759, published 19 July 2009. Crossref's record of DOI 10.1145/1571941.1572114, read 13 October 2026. The paper itself was not read.

How a hybrid retriever works

The problem	Each list misses what the other finds: the figure beside a rare name, the paraphrase
The method	Run both searches; the first candidates chunks of each list go into rrf; cut the fused list to k
No guarantee	A chunk first in one list and absent from the other can be pulled below a chunk both lists have in the middle
You control	The depth of each list (candidates) and the constant
What comes out	One list of k chunks; whether it beats either list alone is a count on questions set aside

Hybrid against each list alone



Key page, 23 questions

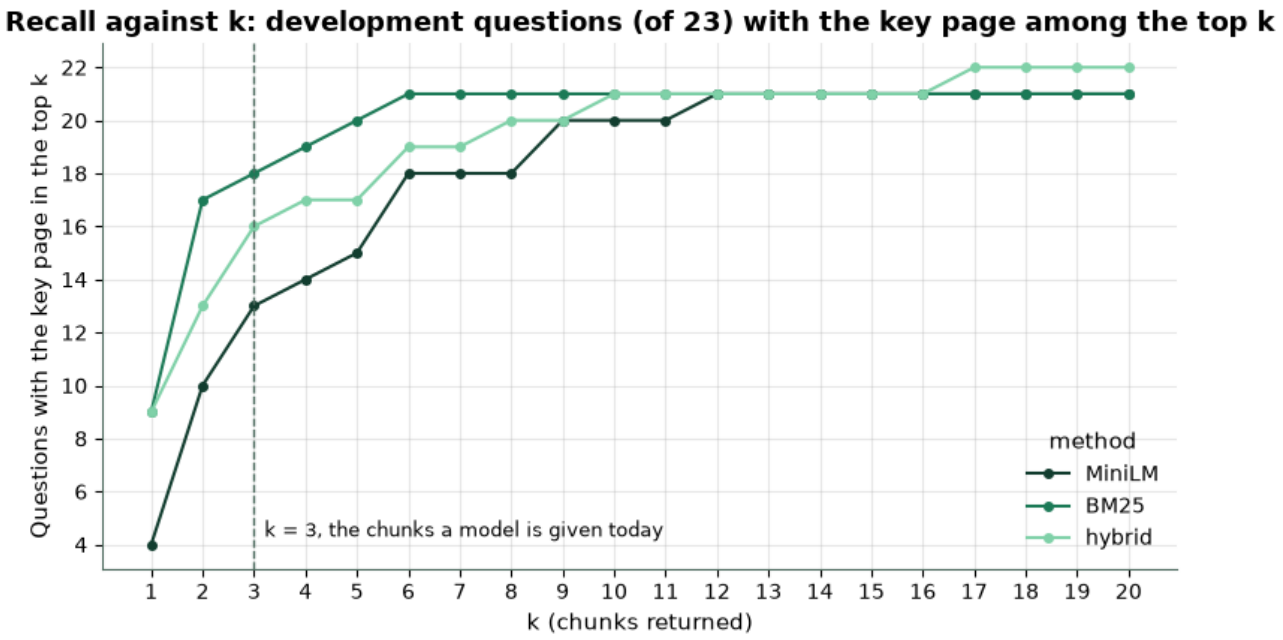
method	top 1	top 3	top 5	top 10
MiniLM	4	13	15	20
BM25	9	18	20	21
hybrid	9	16	17	21

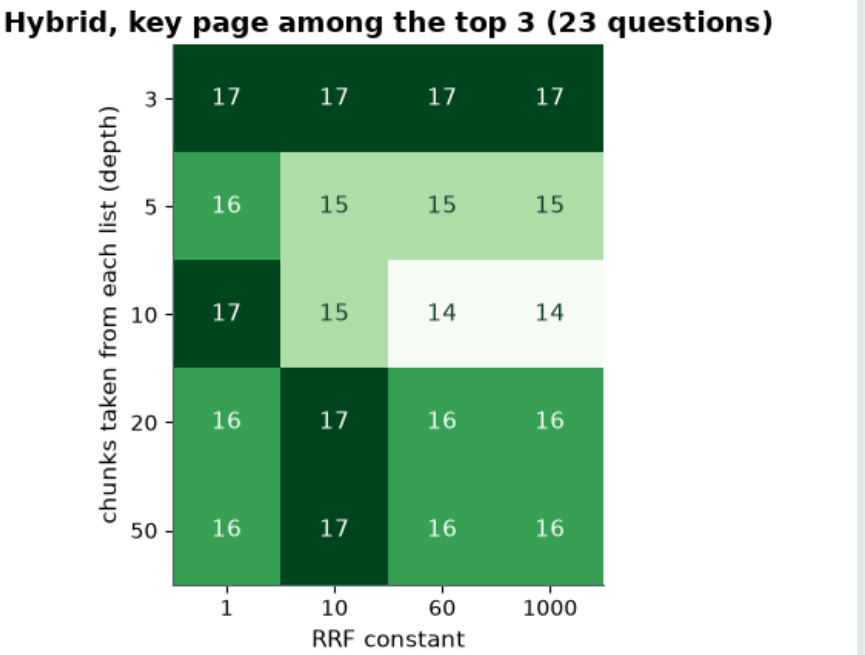
against BM25 at top 3: gains dev21

loses dev07, dev08, dev13

Fusion combines lists; it has no guarantee of beating either. A finding about these questions and these lists.

Recall against k, and fusion's two settings together



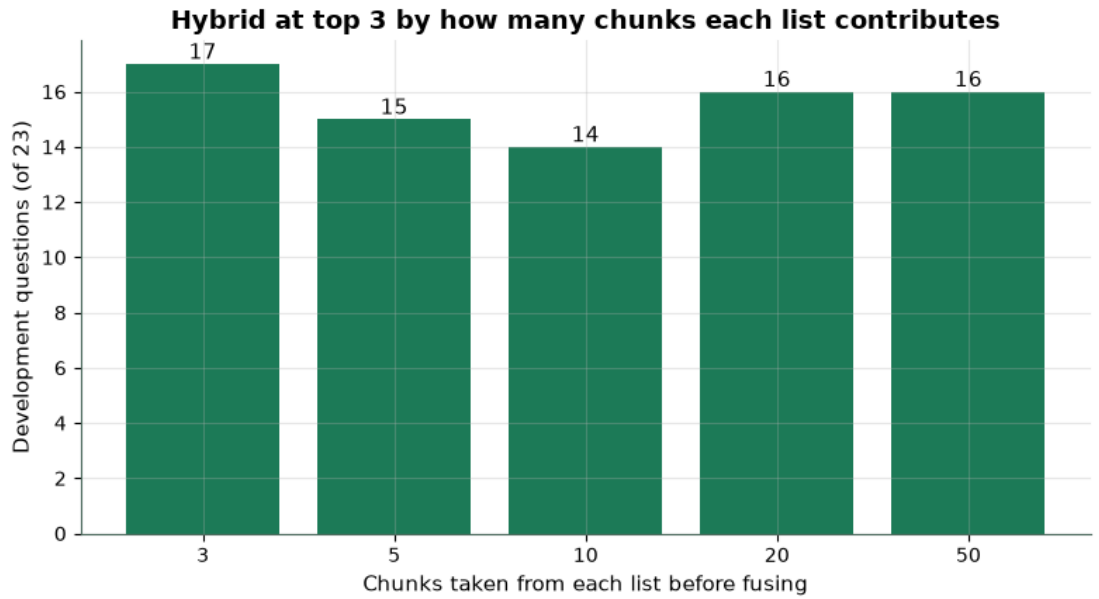


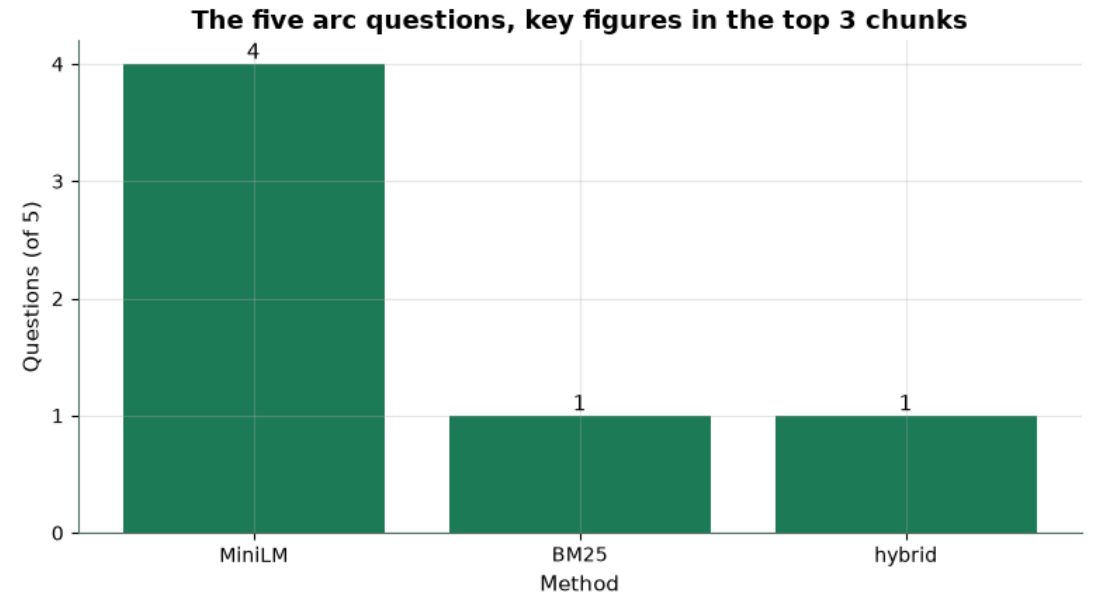
At k = 20: MiniLM 21 of 23, BM25 21 of 23, hybrid 22 of 23; the smallest k at that count: MiniLM 12, BM25 6, hybrid 17

Depth x constant, top 3: 14 to 17 of 23 over 20 settings; the constant moves a row by at most one question

Past the flat part of the curve, more chunks add tokens and no hits.

Depth of the lists, and the arc questions





Hybrid hits at top 3 by depth: 3 -> 17, 5 -> 15, 10 -> 14, 20 -> 16, 50 -> 16

Five arc questions, key figures in the top 3 chunks: MiniLM 4, BM25 1, hybrid 1

Which method has more hits depends on the questions asked; five questions is a very small set.

Every setting of today's retrieval, read from the code

Where	Setting	Default	What it controls	The course sets
`bm25_index()`	**`k1`**	`1.5`	how fast a repeated term stops adding weight (the limit is k1 + 1)	the default
`bm25_index()`	**`b`**	`0.75`	how much a text longer than average is discounted (0 none, 1 in full)	the default
`rrf()`	**`c`**	`60`	how much the top places of a list are worth against the places below them	the default (Cormack, Clarke, and Buettcher's 60)
`RagSettings`	**`retrieve`**	`'dense'`	which list the retriever returns: dense, bm25, or hybrid (the two fused)	dense on day 5; the three compared on day 8
`RagSettings`	**`k`**	`3`	how many chunks the model is given	3
`RagSettings`	**`candidates`**	`20`	how many chunks each list contributes before fusion	20 (the depth of 6.6)

Defaults read with inspect.signature and dataclasses.fields, never typed. The three constants stay; day 8 moves retrieve, candidates, and k.

Excel side by side, and Git

Excel side by side (tested in Excel)

`=LN((E2-D2+0.5)/(D2+0.5)+1)`	idf; equal to Python on 9 rows
`=J2*C2*(H2+1)/(C2+H2*(1-I2+I2*F2/G2))`	weight 0.1245 for text A, senior; equal on 9 rows
`=SUMIFS($K$2:$K$10,$A$2:$A$10,"A")`	A 1.1634, B 0.2393, C 0.4410
`one weight in the 10-K: sofr in boyd10k-8-101-1072`	6.4526 = Python (N 1158, n 17, f 3, dl 88)
`=1/(60+B2), then =SUMIFS($D$2:$D$41,$C$2:$C$41,F2)`	fused scores equal rrf for 34 chunks (1e-15)
`=RANK(G2,$G$2:$G$35)`	ties share a place; rrf breaks ties by id: 9 chunks differ

Git: rag_results.csv with 3 retrieval rows, then a fourth run and its diff; 38 tests.

The AI check

The docstring is the prompt. The retrieval code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the length is counted in characters, divided by an average in tokens, so every length penalty is too large.

Code that runs is not the same as code that is right.

What you can do now

- Search by meaning and by keyword differ on exact strings
- Work BM25 by hand; index with bm25_index; sweep k1 and b as a grid
- Count hits by a named rule, and say what each method misses
- Fuse two lists with rrf; hybrid has no guarantee; draw recall against k
- Read a grid for a plateau; which method has more hits depends on the questions

Week 2, Day 2 Reranking with a cross-encoder